

Building Agents

2. Extending Agents with Tools:

Giving AI Agents the Power to Act with Tools

Language models are strong at reasoning, but without tools, they can only talk—not act. Tools extend an agent’s capabilities beyond text, allowing it to interact with real-world systems like APIs, databases, and mathematical engines.

Tools function like a utility belt for agents. Without access to tools, an agent can't check real-time data like the weather, do precise calculations, or retrieve company records. It might guess or hallucinate. With tools, agents can reliably perform tasks and fetch accurate, up-to-date information.

Common types of tools include:

- **Web search APIs** to answer real-time questions
- **Math functions** to ensure precision in calculations
- **APIs and webhooks** to access services like Slack, CRMs, or travel systems
- **Databases** (SQL/NoSQL) to store or retrieve structured data
- **Code execution** (e.g., Python) for transformations, automation, or logic

Early tool integrations used prompt-based cues like “Calling calculator with X and Y,” followed by backend parsing. This approach was fragile and easy to break. A more reliable method is **Function Calling**, where tools are formally defined and models are trained to use them correctly.

To enable Function Calling, two components are needed:

1. **A model trained to recognize tool-use situations**, fill in arguments, and format calls in structured JSON.
2. **An API that can detect and handle tool calls**, returning the result to the model.

For example, when a user asks, “What’s the weather in São Paulo?”, the model sends a `tool_call` to `get_weather(city="Sao Paulo")`. The backend retrieves the data, returns the result, and the model incorporates it into a natural reply.

More advanced developments are emerging. OpenAI’s Operator, Anthropic’s Model Context Protocol (MCP), and Google’s Agent-to-Agent (A2A) initiative aim to standardize how tools are accessed and how agents communicate across systems. These trends

point toward a future of collaborative, multi-agent systems where each agent specializes in a task.

4. Structured Outputs

Structured Outputs: Making AI Responses Actionable

Agents are more powerful when they return structured, machine-readable outputs like JSON instead of just plain text. Structured responses allow agents to integrate with other systems, power automation, and trigger actions in workflows. For example, rather than summarizing a customer call in text, an agent can extract structured fields like:

```
{  
  "issue_type": "login_problem",  
  "urgency": "high",  
  "customer_email": "jane@mail.com"  
}
```

This format can be directly used in ticketing systems, dashboards, or alerts. Free-form text may be readable, but structured data is what enables downstream tools to act.

Why Prompting Isn't Enough

A common early method was to prompt the model to output JSON, such as: “Return the answer as JSON with fields: issue_type, urgency, customer_email.” While this sometimes works, models often return vague or invalid data, like "urgency": "very" or "email": "none found", which causes errors when code tries to use it. LLMs are trained for natural language, not strict type rules.

Output Parsers and Function Calling

To improve reliability, output parsers can validate whether the model's response conforms to a defined schema. If valid, the output is parsed; if not, fallback strategies can be applied.

Function calling provides an even more robust solution. Here, the model is given a JSON schema and asked to call a specific function. The model must output a structured function call object that matches the schema. This guarantees correct format and types, turning the model into something more predictable and usable in software systems.

Modeling Complex Data with Pydantic

When dealing with more complex outputs—like lists of tasks or nested objects—typed JSON becomes essential. Python’s Pydantic library enables schema enforcement through class definitions. For example:

```
class ActionItem(BaseModel):
```

```
    title: str
```

```
    due_date: datetime
```

```
    owner: str
```

```
    status: Literal["open", "closed"]
```

This ensures that agents generate only valid outputs. If the model's response fails validation, the system can retry, rephrase the request, or handle the error safely.

Designing for Reliability

Agents aren’t perfect and may return malformed data. Structured output mechanisms help catch and manage these errors. Using output parsers, schemas, and retry logic builds resilience into AI systems.

Structured outputs are foundational for agents that do more than talk. They enable integration, automation, and reliability—turning language models into powerful components of real-world workflows.

6. Agent State Management

Managing Agent State with State Machines

LLMs are stateless by default—each interaction is isolated, with no memory of prior prompts. But intelligent agents often need context to manage complex tasks. This requires state: a representation of what the agent knows and is doing during a task.

Stateless vs. Stateful Systems

In a stateless system, every prompt is treated independently. LLMs work this way unless given context in the input. In contrast, a stateful system tracks what has already happened, like the items in an online shopping cart. This ability to retain context is essential for agents that take multiple steps or interact with tools across time.

What Is Agent State?

For LLM-based agents, the state includes:

- The original user input
- System instructions

- A message history
- Any tool calls made or pending
- Intermediate results from prior steps

This state is ephemeral—it only exists while the task is running, unless explicitly saved. It functions like working memory, tracking progress and context throughout the execution.

Using State Machines to Manage Execution

Agents can be modeled as state machines. A state machine transitions through defined steps, updating the internal state at each one. For example, steps may include:

- Preparing the prompt
- Calling the LLM
- Checking for tool use
- Executing tools
- Deciding what to do next

Each step receives the current state, processes it, and returns an updated version. Transitions between steps depend on the contents of the state—this makes the workflow conditional and adaptable.

The Execution Loop

The agent's process typically follows a loop:

1. Receive user query → start with an empty state
2. Step 1: Prepare system and user messages
3. Step 2: Run LLM to generate a response
4. Step 3: If tools are requested:
 - Execute tools
 - Update the state with results
 - Repeat step 2
5. If no tools are needed → end the task

This loop builds a structured execution flow, where the agent reasons, acts, and refines its output through multiple steps.

Defining the State Schema

A type system like Python's TypedDict can define the agent's state structure. Fields might include:

- query: the user input
- instructions: system guidance
- messages: chat history
- tool_calls: pending or completed tool interactions

Each function in the agent's logic receives and returns an object matching this schema, ensuring consistency across steps.

Conditional Transitions

Transitions between steps are dynamic. For instance, the agent should only move to a tool execution step if tools were selected. Transition functions examine the current state and decide which step to take next, often updating the state along the way.

8.Short-Term Agent Memory

Giving Agents a Sense of Continuity with Short-Term Memory

LLMs are naturally stateless—they don't remember past prompts or conversations unless that context is explicitly included. For agents to feel coherent and intelligent, they need a simulated form of memory that helps them maintain context throughout an interaction. This is where short-term memory comes into play.

Memory Is a Simulation

Agents don't have real memory. What looks like memory is actually constructed by including previous interactions in the current prompt. This creates the illusion of continuity. The agent doesn't remember; it's simply reminded.

Ephemeral vs. Durable Memory

There are two main types of memory:

- **Ephemeral (In-application):** Exists only during the session. Once the interaction ends, the memory disappears. This is useful for short-term context and is the focus of this discussion.
- **Durable (Persisted):** Stored in databases or vector stores and available across sessions or days. This enables long-term memory but is not covered here.

Short-Term vs. Long-Term Memory

Short-term memory is like a live conversation—tracking what was just said and building on it. Long-term memory is broader, like remembering past conversations or facts shared across sessions. This module emphasizes short-term memory: maintaining context during a single thread of interaction.

Strategies for Simulating Short-Term Memory

1. **Full Conversation History:** Send all previous messages with each new input. Maintains full detail but consumes many tokens.
2. **Sliding Window:** Include only the most recent messages. Reduces token load but may lose earlier context.
3. **Summarization:** Condense earlier messages into a compact summary. Efficient but relies on effective summarization techniques.

Each method balances context preservation with efficiency. Larger prompts can exceed the model's context window or lead to slower and costlier responses.

State vs. Session Memory

State refers to transient execution information—what the agent is doing right now. It disappears once the task ends.

Session memory covers the broader conversation, tracking tools used, decisions made, and prior responses across steps. While state helps with immediate logic, memory helps the agent act coherently over time.

Why Memory Matters

Short-term memory is what makes agents feel responsive and intelligent. It allows agents to:

- Understand ongoing conversations
- Track context across tool calls
- Reflect on earlier messages

Without memory, agents reset with every prompt. With memory, they feel like assistants capable of maintaining a meaningful dialogue.

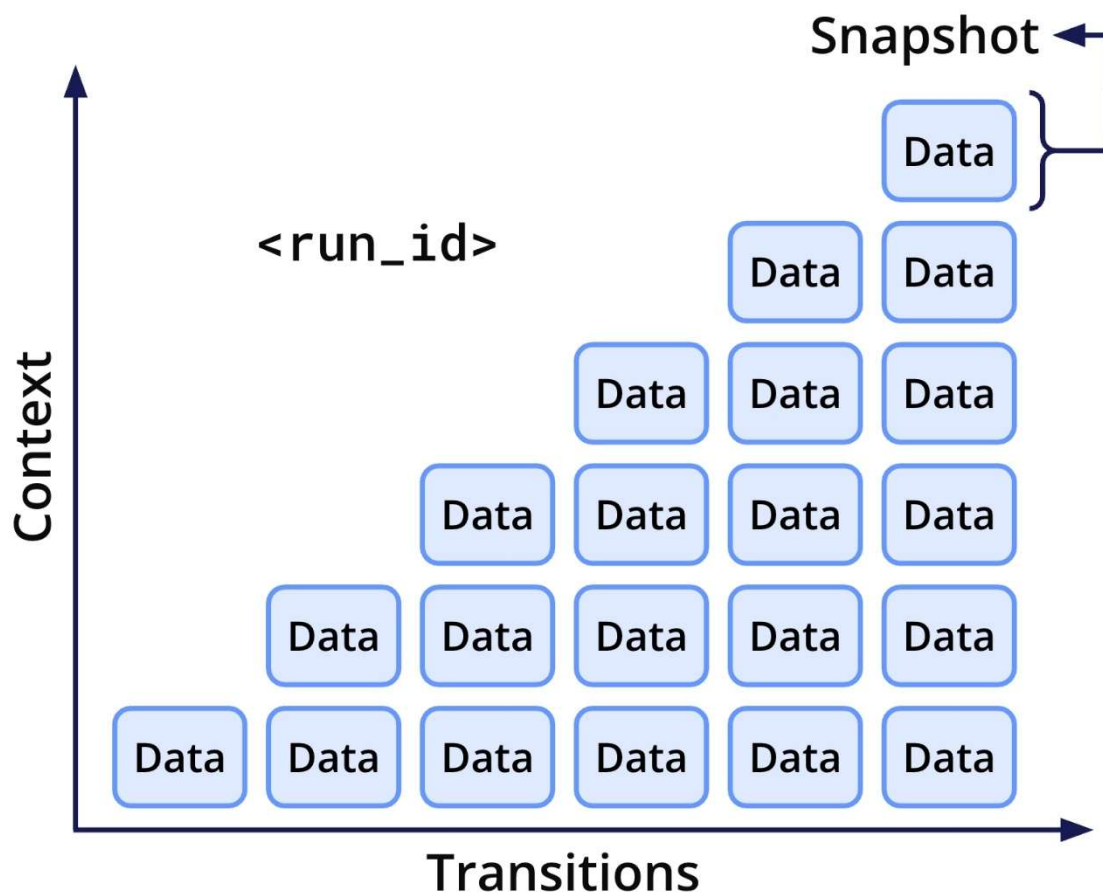
When we talk about memory in the context of agentic systems, we often categorize it into three types: State, Short-term memory, and Long-term memory. It's common to assume these refer to specific durations of time — but in reality, the distinction is more about scope and context, not just how long the information is stored.

Let's break down each one:

State

The state of an agent refers to everything it knows during one execution. This includes intermediate decisions, tool calls, environment data, and transition snapshots as the agent moves through a task.

- **Key idea:** State is ephemeral — it exists only for the duration of the execution.
- **Example:** An agent answering a user question may keep track of which parts of the document it's consulted so far. That's execution state.

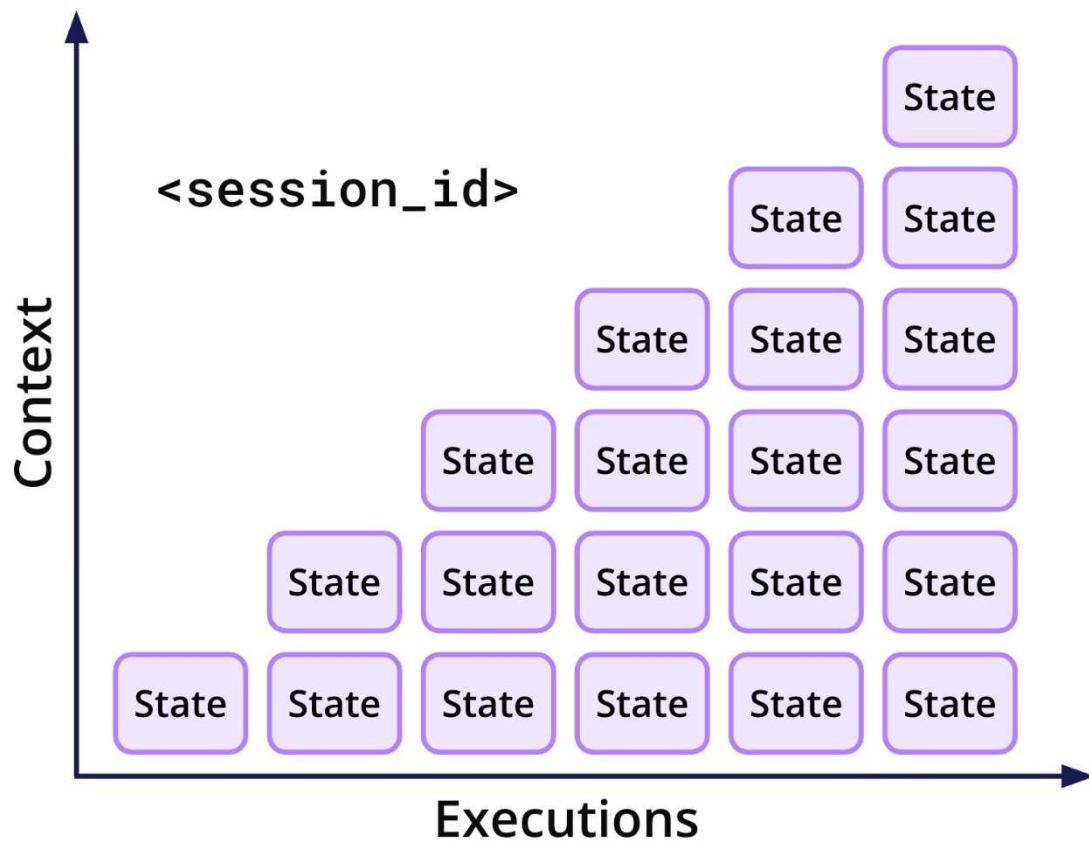


State: Context vs Transitions

Short-term Memory

Short-term memory allows the agent to maintain coherence and context across multiple executions within a single session. It helps the agent remember past interactions while the user is still engaged, without preserving full execution state.

- **Scoped by:** Typically a session ID.
- **Used for:** Dialogue continuity, task follow-up, context threading.
- **Example:** In a multi-turn conversation, the agent remembers what was asked in previous turns and can reference earlier parts of the discussion.

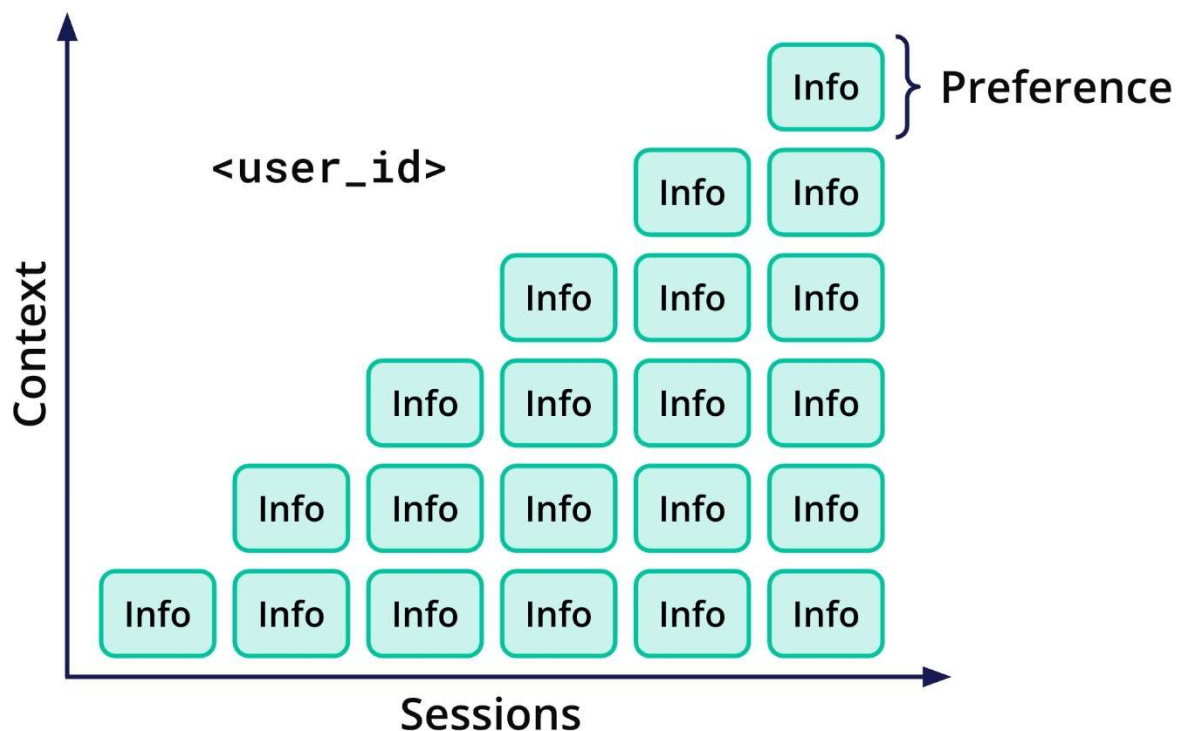


Short-Term Memory: Context vs Executions

Long-Term Memory: Memory Across Sessions

Long-term memory persists key information across sessions and executions, often personalized to the user. It doesn't retain every detail, but stores essential facts, preferences, and experiences to support personalization and learning over time.

- **Scoped by:** Typically a user ID.
- **Used for:** Personalization, knowledge accumulation, remembering user goals and behaviors.
- **Example:** The agent recalls that the user prefers concise summaries or that they're working on a project about climate policy.



Long-Term Memory: Context vs Sessions

10. External Tools and APIs

Connecting Agents to the Real World with External APIs

Agents become truly useful when they stop just reasoning and start acting — interacting with real systems like flight booking services, messaging platforms, or financial data feeds. This shift happens through external APIs, which bridge the gap between an agent's internal reasoning and real-world actions.

Why External APIs Matter

An LLM alone can't access real-time data. Questions like “What’s the cheapest flight to New York this weekend?” require external data sources. Similarly, to send alerts via Slack or trigger actions in other software, agents must call external APIs. These integrations turn agents into doers, capable of meaningful interactions beyond language.

What APIs Really Are

An API (Application Programming Interface) is a structured way for different software systems to communicate. It’s like placing an order at a restaurant: the agent formats the request, sends it to the API (the kitchen), and receives a response (the meal). Most APIs

use HTTP and REST, where the agent sends a request to a URL and receives structured data, often in JSON.

Working with Live Data

External APIs return dynamic, real-time data. The same request might yield different results depending on when it's sent. This makes agents more responsive and context-aware, but it also introduces variability. Outputs may change, tests may become unstable, and behaviors may differ across runs. The benefit is usefulness, but at the cost of predictability.

Handling Authentication

APIs typically require credentials. The simplest form is an API key—a string included in each request. For more secure or user-specific access, OAuth is used. OAuth involves redirecting users to authorize access, receiving a token, and using it securely. Agents interacting with calendars, emails, or sensitive user data must handle authentication correctly.

Dealing with Failures

API calls can fail for many reasons: timeouts, rate limits, unexpected responses. Agents must be resilient—able to retry, fall back, or escalate issues when something goes wrong. This is where agents grow from scripts into autonomous, fault-tolerant systems.

Ethical and Observability Considerations

Real-world actions carry risks. An agent might access private data, make purchases, or delete content. That's why observability is essential—tracking what the agent does, logging actions, and understanding outcomes. Responsible agent design includes safeguards, access controls, and monitoring tools.

Model Context Protocol (MCP)

All our agents so far use direct integrations with their tools (like OpenAI Function Calling or simple API wrappers). But as you go deeper in the world of AI agents, it's important to know about **Model Context Protocol (MCP)** — because it's shaping the future of how agents talk to tools.

Why MCP?

Think about USB-C. Before it came along, every device — phone, camera, laptop — had its own custom cable. Charging and data transfer were clunky, fragile, and inconsistent.

MCP is trying to do for **AI tool use** what USB-C did for device connections.

Right now, every AI model, agent framework, and API speaks a slightly different "language." If you build an agent that uses one model's tools, it doesn't easily work with another model or vendor. Tool use is fragile and non-standard.

MCP provides a **universal connector** — a standard protocol that makes tool use more:

- **Consistent** across models and vendors
- **Composable** across tools and agents
- **Monitorable** and **auditable** for safety and governance

Who's Behind It?

MCP was proposed and is actively developed by **Anthropic**, with contributions from the broader open AI community. It's an **open standard**, meaning it's not tied to any one company.

It's already supported by Anthropic's **Claude models**, and other vendors and frameworks (like **LangChain** and **LangGraph**) are starting to add support.

How It Works

At its core, MCP defines a way for models to:

1. **Express tool calls** — in a standard message format
2. **Receive tool results** — again, in a standard way
3. **Stream interactions** — so agents can handle multi-step workflows dynamically

It separates:

- The model's **reasoning** and **intent** ("I want to use tool X with arguments Y")
- From the system's actual **execution** of tools (calling APIs, handling authentication, retries, logging, etc)

This makes it much easier to build **robust, modular** agent systems.

Transport: SSE is Out, Streamable HTTP is In

Early versions of MCP used **Server-Sent Events (SSE)** for streaming interactions.

But as of the [March 2025 specification\(opens in a new tab\)](#), **Streamable HTTP** is the preferred transport:

- More flexible
- Better for multimodal content
- Easier to adopt in modern architectures

If you see older tutorials mentioning SSE, now you know why it's deprecated.

Why This Matters for You

Although it's new and not yet supported everywhere, you'll likely see MCP popping up lots of places. It's fast becoming the standard for tool-augmented AI. Some places you may see it in the future are:

- **Custom agents**
- **Cross-model agents** (OpenAI + Anthropic + others)
- **Enterprise tool orchestration**
- **Multi-agent systems**

12. Web Search Agents

Teaching Agents to Search the Web and Stay Grounded

Not all useful information is accessible through structured APIs. Details about recent product launches, user-reported bugs, or newly published articles often exist only in unstructured web content. To handle this, agents must be able to search the web, interpret results, and ground their responses in external evidence.

Why Use Web Search?

When asked, "What did Nvidia announce at the last GTC event?", an agent can't rely on training data alone—it might be outdated. The answer could be in a news article, press release, or tweet. Instead of hallucinating, the agent can retrieve real-time information by querying a search engine, just like a human would.

Web Search vs. Web Browsing

It's important to distinguish between web browsing and web search. This module focuses on web search, not browsing web pages or interacting with dynamic content. Web search means sending a query to a search API (like Google or Bing) and receiving results that include titles, snippets, and URLs.

This lightweight approach allows agents to gather high-level, relevant information quickly and reliably.

Grounding Responses with Evidence

LLMs tend to produce confident responses, even when they're incorrect. Web search enables grounding—linking responses to real, verifiable sources. For example, an agent might say, "According to TechCrunch, Nvidia announced a new AI superchip at GTC 2025." Citing sources builds trust and reduces hallucination.

Common Web Search APIs

Several search APIs can be integrated with agents:

- **SerpAPI:** Emulates Google searches, includes images, maps, and more.
- **Tavily:** Clean, LLM-optimized content returns.
- **Bing Search API:** Offers robust and scalable access to search data.

Each one delivers structured JSON search results, but the challenge lies in interpreting and using those results effectively.

Making Sense of Results

Agents must read and evaluate search results to find relevant information. A good agent might:

- Read the top few snippets
- Select the most relevant one
- Summarize it into a response
- Cite the source clearly

This process can be guided by prompt engineering—encouraging the agent to use citations, avoid outdated links, or respond only when confident.

Dealing with Noise and Misinformation

The web isn't perfect. Results can be misleading, outdated, or irrelevant. This introduces noise into the agent's decision-making. Builders must design agents to filter information, question sources, and know when to avoid using live search results.

14. Interacting with Databases

Teaching Agents to Retrieve and Update Structured Data

Agents gain a whole new level of usefulness when they can access and modify private, structured data — like internal records, ticketing systems, or customer databases.

Unlike public web data or APIs, this kind of information lives within an organization and is often stored in structured formats that require precise interaction.

Structured Retrieval in Action

If an agent is asked, “What's the status of ticket #3042?”, it can't look this up with a web search. The answer likely lives in a relational database or CRM. Similarly, if the agent completes a task, it should update the relevant system — not just confirm success in

chat, but actually write a new entry to a database. Agents that interact with structured data can perform real work, not just generate text.

Understanding Databases

- **Relational databases** (e.g., PostgreSQL, MySQL) store data in tables with defined schemas — perfect for agent interactions that require structured queries.
- **NoSQL databases** (e.g., MongoDB) offer flexibility but are less commonly used for direct agent integration.
- **Vector databases** (e.g., Chroma, Pinecone) store embeddings for semantic search, enabling similarity-based retrieval, a key part of many RAG (Retrieval-Augmented Generation) workflows.

For now, the focus is on relational databases and SQL.

text2SQL: Translating Language into Queries

Agents can translate natural language into SQL — a practice known as text2SQL. For example:

Input: “How many users signed up this week?”

Output: `SELECT COUNT(*) FROM users WHERE signup_date >= '2025-07-01'`

This requires the agent to:

- Understand database schemas
- Handle filters and date ranges
- Avoid dangerous queries (e.g., accidental DELETE)

To prevent critical errors, real-world agents must be wrapped with guardrails, including query validation and human approvals.

Using Vector Databases for Semantic Search

Some questions don’t have exact answers in a database. For example, finding documents similar in meaning to a question. Vector databases handle this by comparing embeddings and returning the most semantically relevant content — not just exact matches.

RAG Doesn’t Require Vector DBs

RAG is a pattern, not a tool. Retrieval can use vectors, SQL, keyword search, or combinations of all three. Effective agents often combine approaches — using SQL to

filter and vector search to rank results semantically. This hybrid model gives agents the ability to reason over both the structure and meaning of data.

16. Agentic retrieval Augmented Generation

From Passive Lookup to Active Reasoning: Building Agentic RAG

Basic Retrieval-Augmented Generation (RAG) systems follow a simple pattern: accept a query, retrieve a few documents, and generate an answer. This works for straightforward questions but falls short in more complex scenarios. Agentic RAG enhances this by giving agents the ability to reflect, retry, and refine — transforming retrieval from a static tool into an active part of the agent’s reasoning loop.

Beyond One-Shot Retrieval

Traditional RAG does not evaluate the usefulness of what it retrieves. It retrieves once and moves on. Agentic RAG introduces a feedback loop:

- The agent assesses whether the documents are relevant
- It identifies missing information
- It reformulates queries if needed
- It repeats retrieval and adjusts strategies before answering

This makes the process iterative and intelligent, not one-and-done.

Case Study: Zillow Offers Failure

Search results on Zillow’s failed home-buying program, which ran from 2018 to 2021, reveal multiple contributing factors, including flawed algorithmic assumptions, unpredictable market behavior, and internal business pressures. (See resources below for more on these factors.)

A simple RAG agent might summarize that Zillow’s housing algorithm failed due to unpredictable markets. An agentic RAG agent, however, would:

- Notice gaps in the initial retrieval
- Reflect on what’s missing (e.g., algorithmic assumptions, feedback loops)
- Reformulate the query to focus on technical flaws and business strategy
- Retrieve better data and produce a layered explanation

The final answer becomes nuanced, grounded, and explanatory — not just a surface-level summary.

Planning and Flexible Workflows

Agentic RAG agents don't retrieve blindly. They plan:

- When to retrieve (or skip retrieval entirely)
- What to retrieve first (e.g., definitions, background, evidence)
- How to integrate retrieved context across multiple steps

Agents also inspect their results:

- Do they match query keywords?
- Do they answer “what” or “why” directly?
- Can each claim be backed by a specific source?

This self-monitoring guides whether to retry, pivot tools, or adapt strategy.

Reflection and Retry Strategies

Agents can score retrieved passages or summarize what's known vs. unknown before answering. These heuristics, often guided by prompt engineering or scoring models, help determine confidence and whether another retrieval round is necessary.

Frameworks may include metrics like **faithfulness** and **relevancy**, but even without them, the agent learns to reflect on its process — a major leap from static pipelines.

18. Long Term Memory:

Long-Term Memory: Helping Agents Build Relationships Over Time

While short-term memory allows agents to stay coherent within a single session, long-term memory enables them to recall facts, experiences, and preferences across sessions. This transforms agents from stateless tools into persistent assistants, capable of personalization and improvement over time.

Why Long-Term Memory Matters

Without memory, agents start fresh every time. But for agents designed as tutors, assistants, or companions, continuity is essential. Long-term memory allows them to:

- Personalize responses using stored knowledge
- Avoid repeating questions or instructions
- Learn from past interactions to improve over time

This makes interactions feel more intelligent and human-like.

Three Types of Long-Term Memory

Long-term memory typically falls into three categories:

1. **Semantic Memory:** Stores factual information learned from users, such as names, preferences, tools used, or product choices. These facts are often stored in structured formats (e.g., user profiles or key-value stores) and retrieved using similarity or context cues.
2. **Episodic Memory:** Captures past interactions and events. If an agent successfully helped with a database connection in a previous session, it can reference that to assist again. These memories support few-shot learning by recalling helpful examples from the past.
3. **Procedural Memory:** Encodes behavioral adaptations. For instance, if a user consistently prefers formal tone, the agent can adjust its style accordingly. These changes often affect internal prompts or dynamic rules, helping the agent improve how it helps, not just what it knows.

Storing and Retrieving Memory

Long-term memory can be stored in:

- Vector databases (e.g., Chroma, Pinecone)
- Relational databases (e.g., PostgreSQL)
- Document stores (e.g., MongoDB)

Memory retrieval is scoped by user, team, or globally. For example:

- **User-scoped:** Tied to a specific identity (e.g., email)
- **Team-scoped:** Shared across a group
- **Global-scoped:** Learned patterns across users

Triggers for retrieval might include semantic similarity, tags, time, or session context. Retrieved memories can be injected into system prompts, background context, or passed as tool parameters.

Challenges and Best Practices

Good memory design balances usefulness with responsibility. Key practices include:

1. **Selectivity:** Only store what's relevant; not every message needs to be remembered.
2. **Expiry:** Use time-to-live (TTL) settings to automatically retire outdated information.

3. **Separation:** Keep memory namespaces clear to prevent overlap between users or teams.
4. **Balance:** Consider privacy and performance when deciding what to store and how it's accessed.

20. Agent Evaluation

Evaluating Agents: How to Measure What Matters

Building an agent that can call tools, track state, use memory, and route queries is a major accomplishment — but it's only half the story. The real question is: does it work well, safely, and efficiently? Agent evaluation provides the feedback loop necessary to understand, improve, and trust your system.

Why Evaluation Is Essential

Without evaluation, agent behavior can drift unnoticed. An agent might start making incorrect tool calls, wasting tokens, or even taking harmful actions. For example, rather than alerting a sales rep about a disengaged customer, it might delete their records — a critical mistake. Evaluation ensures visibility into performance and safety, enabling iteration and progress.

Four Dimensions of Evaluation

Effective evaluation looks across multiple dimensions:

- **Task Completion:** Did the agent achieve its goal? How many steps did it take? Was human intervention needed?
- **Quality Control:** Was the output in the correct format? Did it follow prompt instructions and use the provided context?
- **Tool Interaction:** Did the agent choose appropriate tools and use valid arguments?
- **System Metrics:** How efficient was the run in terms of time, token usage, and failure rates?

Together, these give a comprehensive view of how the agent performs.

Three Evaluation Strategies

1. **Final Response Evaluation:** Judges the end result only. This black-box method is great for end-to-end testing and can use an LLM-as-judge to assess success. However, it lacks insight into how the result was produced.

2. **Single-Step Evaluation:** Focuses on one decision at a time, like selecting a tool or formatting a query. It's fast and ideal for debugging, but doesn't capture the agent's overall effectiveness.
3. **Trajectory Evaluation:** Reviews the full execution path — tool calls, order, arguments, and results. This offers the richest insight, especially for complex workflows, but requires more setup and analysis.

What You Need for Evaluation

No matter the method, four core components are required:

- **Inputs:** User queries, tool options, memory, and instructions
- **Outputs:** Agent responses and tool call sequences
- **Reference Data:** Ground truth answers or expected actions
- **Evaluators:** Rules, heuristics, or models (like an LLM) to assess success

Sometimes exact reference data exists. Other times, evaluations rely on approximations and best-guess heuristics. The key is consistency — enabling meaningful comparisons over time.

The Challenge of Evaluation

Agents are dynamic and their tasks can be subjective. There's no universal metric for success, and overfitting to narrow criteria is a risk. Evaluation needs to go beyond correctness, covering efficiency, robustness, and user experience.